Product Requirements Document (PRD)
Project Name: ModernOS Content Relay (MCR)
Document Status: Draft / V1.0
Target Release: Sprint 3

1. Objective

To build a fully automated, zero-touch media pipeline that ingests short-form video content (Reels) shared privately via direct message, algorithmically enhances the content for virality using multimodal AI, and autonomously republishes the asset to a public-facing social media account.

2. Problem Statement

Manually downloading, editing, writing captions for, and republishing curated content across multiple Instagram accounts is a high-friction, time-consuming process. Relying entirely on official APIs is impossible without converting all accounts to Business profiles and losing access to native DM sharing. A hybrid approach is required to bridge private curation and public broadcasting seamlessly.

3. Scope

3.1 In-Scope

* Edge Ingestion: Browser-based automation to securely read incoming DMs on a proxy account and extract media URLs.
* Cloud Processing Engine: Serverless ingestion and storage of raw video files.
* AI Enhancement: Native multimodal analysis of video context to generate dynamic text overlays (hooks) and SEO-optimized captions.
* Media Factory: Cloud-based video rendering to burn text overlays into the video file.
* Edge Publishing: Device-based UI automation to natively upload and publish the final asset to a public account.
* State Management: Real-time database tracking to prevent duplicate processing and manage the job queue.

3.2 Out-of-Scope

* Direct API integration via Meta Developer Platform (to maintain standard account statuses).
* Cross-platform posting (e.g., TikTok, YouTube Shorts) for the V1 launch.
* Automated replying or community management on the published posts.

4. System Architecture

The system operates on a "Split-Stack" paradigm, utilizing a local Mac Mini M4 for human-mimicry tasks (The Edge) and Google Cloud Platform for heavy lifting (The Cloud).

Layer	Environment	Technology	Function
Ingestion	Edge (Mac Mini)	Playwright (Python)	Scrapes DM, extracts URL, prevents duplicate reads.
Orchestration	Edge / Cloud	n8n / Webhooks	Routes URLs to downloaders, pushes files to GCS.
Storage	Cloud (GCP)	Cloud Storage	Hosts raw-input and processed-output buckets.
Intelligence	Cloud (GCP)	Vertex AI (Gemini)	Analyzes video; outputs JSON hook, caption, hashtags.

Rendering	Cloud (GCP)	Cloud Run (FFmpeg)	Containerized processing to burn text overlays into video.
State	Cloud (GCP)	Firestore	Tracks job status (analyzed, ready_to_upload, published).
Publishing	Edge (Mac Mini)	Appium (Android/UI)	Listens to Firestore, downloads final video, executes UI clicks to post.

5. Functional Requirements

F1. Stealth DM Extraction

* The system must load a persistent, pre-authenticated browser session.
* It must navigate to the specific chat thread and extract the most recent instagram.com/reel/ URL.
* It must maintain a local ledger of processed URLs to prevent duplicate loops.

F2. Cloud Video Ingestion

* Upon receiving a valid URL webhook, the system must download the .mp4 payload.
* The payload must be written to the raw-input Cloud Storage bucket.

F3. Multimodal AI Analysis

* A Cloud Function must trigger on file creation in the raw-input bucket.
* The function must pass the video URI to Gemini 1.5 Pro to generate a 3-5 word hook, a caption, and 3-5 contextual hashtags.
* The output JSON must be written to the Firestore job_queue.

F4. Headless Video Rendering

* The rendering service must pull the raw video and the AI-generated hook text.
* It must apply the text as a centralized, high-contrast overlay on the video.
* It must preserve the original audio track without degradation.
* The final file must be uploaded to the processed-output bucket.

F5. Native UI Publishing

* A local daemon must listen for ready_to_upload status updates in Firestore.
* Upon trigger, it must download the final .mp4.
* It must launch the Android emulator, push the video to the device gallery, and execute the exact UI taps required to publish a Reel, including pasting the AI-generated caption.

6. Non-Functional Requirements (NFRs)

* Anti-Ban Mechanisms: * Browser automation must use playwright-stealth to mask web driver signatures.
  * Appium automation must implement randomized human_sleep() intervals between UI interactions (1.5s to 4.2s).
  * No automated login sequences are permitted; all scripts must rely on persistent session tokens.
* Performance: The cloud processing phase (Upload -> AI Analysis -> FFmpeg Render -> Ready Status) must complete in under 45 seconds per video.
* Scalability: The Cloud Run rendering service must be able to process at least 10 concurrent video jobs asynchronously.
* Error Handling: If a UI element changes (e.g., Instagram moves the 'Share' button), the Edge Publisher must safely fail, log the error locally, and halt execution without crashing the background daemon.

7. Success Metrics

* Zero-Touch Rate: 95% of forwarded Reels are successfully published to the public account without manual intervention.
* Time-to-Publish: End-to-end pipeline execution (from DM receipt to public post) completes in under 5 minutes.
* Account Health: 0 account flags, action blocks, or shadowbans incurred on either the Proxy or Public accounts over a 30-day period.

Would you like to lock this PRD in and begin building the n8n orchestration webhook outlined in Epic 2, or start by generating the Playwright session state?

Here is the complete, concrete Jira project plan for the ModernOS Content Relay (MCR).
I have structured this into 5 Core Epics that follow the chronological pipeline of the data: from scraping the DM, to orchestrating the download, analyzing via AI, rendering the video, and finally, publishing.
You can copy and paste these directly into your Jira backlog.

🚀 Project: ModernOS Content Relay (MCR)

🟦 Epic 1: Cloud Foundation & IAM (GCP)

Objective: Establish the secure, scalable Google Cloud environment that will act as the backend factory for the entire pipeline.

* MCR-101: Initialize GCP Project & Enable APIs
  * Description: Create the modernos-relay project. Enable Vertex AI, Cloud Run, Cloud Functions, Firestore, and Cloud Build APIs.
  * Acceptance Criteria: gcloud CLI is authenticated locally. Project exists and billing is linked.
* MCR-102: Provision Storage & Database Layers
  * Description: Create GCS buckets for mcr-raw-input and mcr-processed-output. Initialize Firestore in Native mode with a job_queue collection.
  * Acceptance Criteria: Both buckets exist in us-central1. Firestore is active and accessible via console.
* MCR-103: Configure Edge-to-Cloud Security (Service Accounts)

  * Description: Create the modernos-edge-agent IAM Service Account. Assign Storage Admin, Datastore User, and Vertex AI User roles.
  * Acceptance Criteria: JSON key file is generated and securely stored on the Mac Mini.

🟦 Epic 2: The Ingestion Edge (Scraper & n8n)

Objective: Safely extract incoming Reel links from the Proxy account's DMs and push them to the cloud without triggering Meta's bot detection.

* MCR-201: Develop Playwright Stealth DM Scraper
  * Description: Build a Python script using playwright-stealth that loads a saved session state, navigates to Instagram DMs, and extracts the latest Reel URL from the target chat.
  * Acceptance Criteria: Script successfully prints the raw instagram.com/reel/... URL without triggering a login prompt.
* MCR-202: Implement Local Deduplication Database
  * Description: Add a lightweight SQLite or .txt log to the scraper to record processed URLs.
  * Acceptance Criteria: Script ignores URLs it has already scraped and only forwards genuinely new DMs.
* MCR-203: Build n8n Orchestration Webhook
  * Description: Create an n8n workflow that receives the URL from the scraper via webhook, uses a downloading API/tool (like yt-dlp) to fetch the .mp4, and uploads it to the mcr-raw-input GCS bucket.
  * Acceptance Criteria: Firing the Playwright script results in a .mp4 appearing in the GCP bucket automatically.

🟦 Epic 3: The Intelligence Layer (Vertex AI)

Objective: Use multimodal AI to "watch" the video and generate viral metadata (hooks, captions) entirely hands-free.

* MCR-301: Deploy Video Ingestion Cloud Function
  * Description: Write a Gen 2 Cloud Function triggered by google.cloud.storage.object.v1.finalized on the input bucket.
  * Acceptance Criteria: Uploading a video to GCS automatically triggers the function.
* MCR-302: Integrate Gemini 1.5 Pro Multimodal Prompting
  * Description: Add Vertex AI SDK to the Cloud Function. Pass the GCS URI to Gemini with the "Viral Strategist" prompt.
  * Acceptance Criteria: Gemini successfully analyzes the video without needing separate audio extraction and returns a JSON payload.
* MCR-303: Sync AI Metadata to Firestore
  * Description: Parse the Gemini JSON output and write a new document to the job_queue collection containing the hook_text, caption, and status analyzed.
  * Acceptance Criteria: Firestore document is created instantly upon Gemini analysis completion.

🟦 Epic 4: The Media Factory (Cloud Run)

Objective: Automatically edit the raw video by burning in the AI-generated text hooks and optimizing the format for Instagram.

* MCR-401: Create Dockerized Media Factory Container
  * Description: Write a Dockerfile that installs Python 3.11, ffmpeg, and standard fonts. Setup a basic Flask app.

  * Acceptance Criteria: Container builds successfully locally.
* MCR-402: Implement FFmpeg Text Burn-in Logic
  * Description: Write the Python subprocess logic to download the video from GCS, apply the hook_text as a visual overlay using ffmpeg filters, and upload the result to mcr-processed-output.
  * Acceptance Criteria: Processed video visually displays the text overlay in the correct coordinates.
* MCR-403: Deploy Factory & Link to Firestore Pipeline
  * Description: Deploy the container to Cloud Run. Add a trigger (via Eventarc or a direct HTTP call from the Phase 3 Cloud Function) to initiate the render when the status hits analyzed. Update Firestore to ready_to_upload upon completion.
  * Acceptance Criteria: End-to-end cloud test: Raw video in -> Gemini analysis -> Cloud Run render -> Final video out with Firestore status updated.

🟦 Epic 5: The Publishing Edge (Appium)

Objective: Bring the processed file back to the Mac Mini and physically automate the UI to post it natively to the Public account.

* MCR-501: Configure Local Appium & Emulator Environment
  * Description: Install Appium 2.0, UIAutomator2 driver, and setup an Android Virtual Device (AVD) running the Instagram APK.
  * Acceptance Criteria: appium server runs and can successfully connect to the emulator.
* MCR-502: Develop Appium Native Posting Script
  * Description: Write a Python script that takes a local file path and a caption, pushes the file to the emulator gallery, and clicks through the Instagram UI to publish the Reel.
  * Acceptance Criteria: Script successfully posts a test video to the public account.
* MCR-503: Build Firestore Listener Agent
  * Description: Write a daemon script that uses on_snapshot to listen to the Firestore job_queue. When a document hits ready_to_upload, it downloads the GCS file, triggers the Appium script (MCR-502), and marks the job as published.
  * Acceptance Criteria: The entire pipeline flows from start to finish without manual intervention.

Suggested Sprint Plan (2-Week Sprints)

* Sprint 1: Epics 1 & 2 (Get the infrastructure up and extract the raw data).
* Sprint 2: Epics 3 & 4 (Build the cloud brain and media factory).
* Sprint 3: Epic 5 (Close the loop with automated publishing).

How would you like to proceed? We can jump straight into Sprint 1 and start configuring the n8n webhook and the deduplication logic for the scraper we just built.